AI-Powered CVE Triage

on the Jetson Orin Nano — Part 1: The Idea

`SJSU · Edge AI · Cyber-AI`

An LLM + a few tools that decides which scanner findings are *actually* exploitable.

1 Finding a CVE ≠ triaging it

A scanner (`pip-audit`) cross-checks your `requirements.txt` against the CVE database:

```
$ pip-audit -r requirements.txt
Found 33 known vulnerabilities in 4 packages.
  requests 2.19.1  CVE-2018-18074  leak Proxy-Authorization on redirect
  jinja2   2.10    CVE-2019-10906  str.format_map sandbox escape
  pyyaml   5.3     CVE-2020-1747   yaml.load arbitrary code execution  ...
```

Every line is *technically* true. The real question for the engineer is:

> **"Is this CVE actually reachable from *our* code — or a false alarm?"**

2 Three buckets, one hard decision

Bucket	Meaning	Cost of error
Exploitable here	Patch now — real bug.	High if missed
Not exploitable	Suppress / defer to next upgrade.	High if misclassified
Inconclusive	Needs a human.	Bounded

Humans triage ~30 findings/hour. We compress NVIDIA's production vulnerability-analysis blueprint into **~600 lines of single-file Python** on one Jetson.

3 Why an LLM (not a regex)

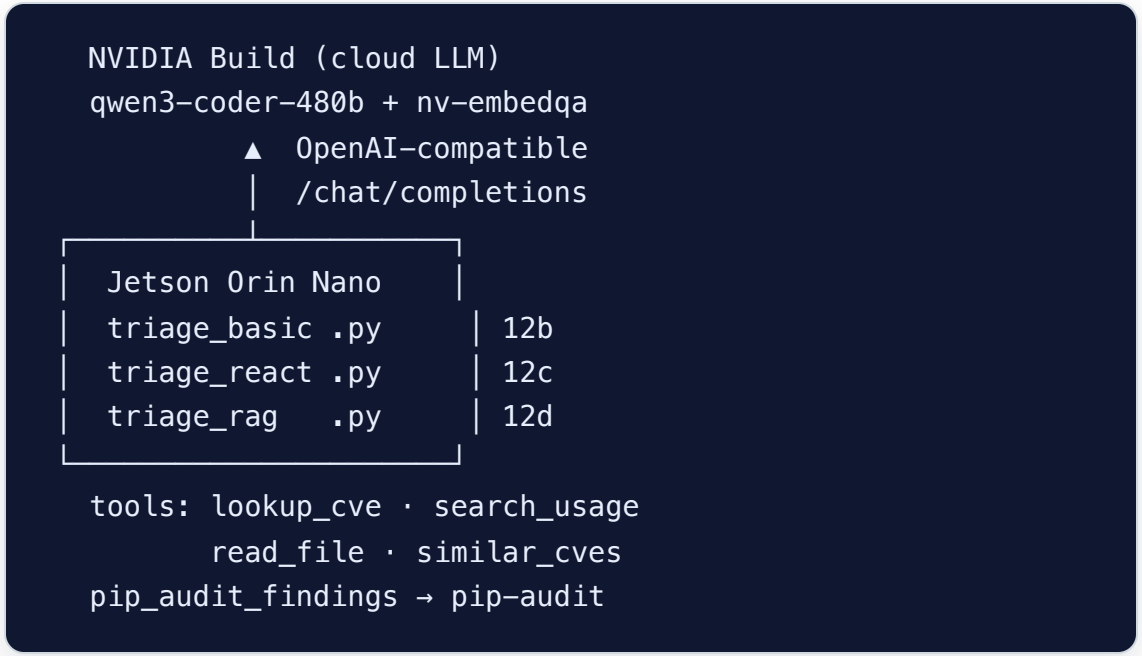
The scanner knows the **facts** (version X has CVE Y). It can't answer the **semantic** question:
does our code even call the vulnerable function, with attacker-controlled input?

A coding LLM (`qwen/qwen3-coder-480b`) on NVIDIA Build can:

1. **Read** the CVE → identify the vulnerable pattern (`yaml.load(untrusted)`).
2. **Search** our codebase for that pattern (a tool we hand it).
3. **Reason** about context — is the input really attacker-controlled?
4. **Emit** a JSON verdict downstream CI can ingest.

> The model **never sees the whole codebase** — it pulls only the bytes it needs via tools.

4 The architecture



3 entrypoints · 4 tools · 1 sample project.

We cut from the blueprint:

- Morpheus pipeline → one `for` loop
- LangGraph → OpenAI tool-calling
- Triton → NVIDIA Build endpoints
- Milvus → in-memory cosine over ~12 rows
- Docker/Helm → `python triage_basic.py`

Kept: an LLM with tools, classifying each finding.

5 The sample project — a triage puzzle

`app.py` deliberately exercises three distinct shapes:

```python
import jinja2, requests
_STATUS_TEMPLATE = jinja2.Template("Status for {{ url }}: {{ status }}")  # constant!

def fetch_status(url: str) -> dict:           # requests: caller-supplied URL
    response = requests.get(url, timeout=5)    # → vulnerable path REACHABLE
    return {"status": response.status_code, "length": len(response.content)}

def render_status(url, status):               # jinja2: only a fixed template
    return _STATUS_TEMPLATE.render(url=url, status=status)   # → NOT reachable
# pyyaml: in requirements.txt, never imported  → dead weight
```

Package	Used?	Expected verdict
`requests`	✅ caller-supplied URL	**Exploitable**
`jinja2`	✅ hard-coded template	**Not exploitable**
`pyyaml`	❌ never imported	**Not exploitable** (dead weight)

6 Run it — inside the container

```
sjsujetsontool shell                        # Jetson AI container; brings in ~/.env.local (NVIDIA_API_KEY)
cd /Developer/edgeAI/edgeLLM/vuln-triage
pip install -r requirements.txt             # openai · httpx · pip-audit
```

Prove the **scanner half** works before adding any LLM:

```
python3 -m pip_audit -r sample_project/requirements.txt --format json --no-deps \
  | jq '.dependencies[] | {name, version, n: (.vulns|length)}'
```

```
{"name": "requests", "version": "2.19.1", "n": 6}
{"name": "jinja2",   "version": "2.10",   "n": 6}
{"name": "pyyaml",   "version": "5.3",    "n": 4}
```

7 The agent loop (lesson 12b)

Hand the model **four tools** as OpenAI JSON schemas, then loop:

```
TOOL_SCHEMAS = [ lookup_cve, pip_audit_findings, search_usage, read_file ]  # JSON schemas

for round in range(MAX_TOOL_ROUNDS):                  # ~6
    resp = client.chat.completions.create(
        model=model, messages=messages,
        tools=TOOL_SCHEMAS, tool_choice="auto")       # model picks a tool
    msg = resp.choices[0].message
    if not msg.tool_calls:                            # no tool → it's the verdict
        return parse_verdict(msg.content)
    for tc in msg.tool_calls:                         # run each tool, feed result back
        result = TOOL_IMPL[tc.function.name](**json.loads(tc.function.arguments))
        messages.append({"role": "tool", "tool_call_id": tc.id, "content": result})
```

One OpenAI tool-calling loop, zero frameworks. 12c rewrites it as a manual ReAct loop; 12d adds retrieval.

8 The four-part series

Part	What you learn	Code
12 (here)	Problem · sample data · architecture	—
12b	Single-turn OpenAI tool-calling	`triage_basic.py`
12c	Manual ReAct loop (any chat model)	`triage_react.py`
12d	ReAct + embedding retrieval (agentic RAG)	`triage_rag.py`

Full lesson → lkk688.github.io/edgeAI/curriculum/12_vulnerability_triage_intro